

# Deep Learning for Economics and Finance

From basic ML through DNNs to sequence models for economic time series

Simon Scheidegger

HEC, University of Lausanne

# Course Overview

## Block 1: Discrete-time methods

- ▶ Intro to ML & Deep Learning
- ▶ Deep Equilibrium Nets (DEQNs)
- ▶ IRBC, NAS & Loss Normalization
- ▶ OLG, Young's Method & Sequence Space

## Block 2: Agentic AI, continuous-time methods, applications

- ▶ Agentic Programming (AI for Research)
- ▶ PINNs & CT Heterogeneous Agents (HJB+KFE)
- ▶ Surrogates, GPs & Deep Kernels
- ▶ Climate Economics, IAMs & Deep UQ

**All materials:** [https://github.com/sischei/Deep\\_Learning\\_for\\_Solving\\_And\\_Estimating\\_Dynamic\\_Economic\\_Models](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)

# Roadmap of This Lecture

## Machine Learning Foundations

- ▶ What is ML? Supervised, unsupervised, RL
- ▶ The three-step ML recipe

## From Perceptrons to Deep Nets

- ▶ The perceptron and activation functions
- ▶ Multi-layer networks and universal approximation

## Training, Generalization & Practice

- ▶ Losses, gradient descent, backpropagation
- ▶ Regularization, optimizers, software ecosystem

## Sequence Models

- ▶ RNNs, LSTMs, attention, transformers
- ▶ What matters for economic time series

# Learning objectives

**By the end of this lecture you should be able to:**

- ▶ Classify supervised, unsupervised, and reinforcement learning problems and select appropriate methods for economic applications
- ▶ Build neural networks with multiple layers and nonlinear activations, and explain why nonlinearity is essential
- ▶ Implement the three-step ML recipe (model, loss, optimizer) and apply gradient descent with backpropagation
- ▶ Diagnose overfitting versus underfitting and apply regularization techniques to improve generalization
- ▶ Construct sequence models (RNNs, LSTMs, Transformers) to handle economic time series data



# The Rise of Neural Networks

IGN, BUSINESS, BUSINESS 01.25.17 01:00 PM

## DEEPMIND BEATS PROS AT STARCRAFT IN ANOTHER TRIUMPH FOR BOTS



A pro gamer and an AI bot duke it out in the strategy game StarCraft, which has become bed for research on artificial intelligence. STARCRAFT

**IN LONDON LAST** month, a team from Alphabet's UK-based artificial intelligence research unit DeepMind quietly marked a new marker in the contest between humans and computers. On Thursday it revealed the achievement in a three-hour YouTube stream, in which aliens and robots fought to the death.

IGN, BUSINESS, BUSINESS 10.18.17 01:00 PM

## THIS MORE POWERFUL VERSION OF ALPHAGO LEARNS ON ITS OWN



NOAH SHEDDEN FOR WIRED

**AT ONE POINT** during his historic defeat to AlphaGo last year, world champion Go player Lee Sedol abruptly left the room. The bot had played a move that confounded established theories of the game, a moment that came to epitomize the mystery of AlphaGo.

WASH. POST, 21 DECEMBER 2017

## AI beats docs in cancer spotting

A new study provides a fresh example of machine learning as an important diagnostic tool. Paul Biegler reports.



## Deep Learning Software Speeds Up Drug Discovery

Wed, 01/11/2018 - 8:00am 1 Comment by [Kenny Walter](#), Science Reporter - [@RandDMagazine](#)



The long, arduous process of narrowing down millions of chemical compounds to just a select few that can be further developed into mature drugs, may soon be shortened, thanks to new artificial intelligence (AI) software.

# AI Music Generation



**OpenAI MuseNet** can generate 4-minute musical compositions with 10 different instruments, combining styles from country to Mozart to the Beatles.

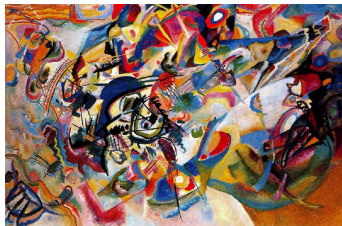
Deep learning enables machines to:

- ▶ Compose music in different genres
- ▶ Combine multiple instrumental tracks
- ▶ Learn long-range structure in sequences

# Neural Style Transfer



Content image



Style image (Kandinsky)



Result

Gatys, Ecker & Bethge (2015): a neural network separates and recombines content and style of images.

# Self-Driving Cars

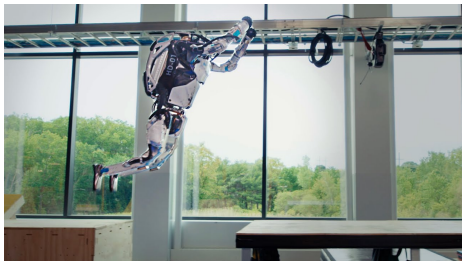


**ALVINN** (1989), one of the first autonomous vehicles, using a simple neural network.



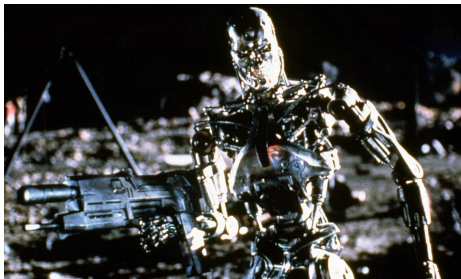
**Waymo** (2017+), deep learning for perception, prediction, and planning.

# Two-Legged Robots



**Boston Dynamics Atlas**

<https://www.bostondynamics.com/atlas>



Science fiction meets reality, deep reinforcement learning enables humanoid locomotion.

# Coloring Old Movies



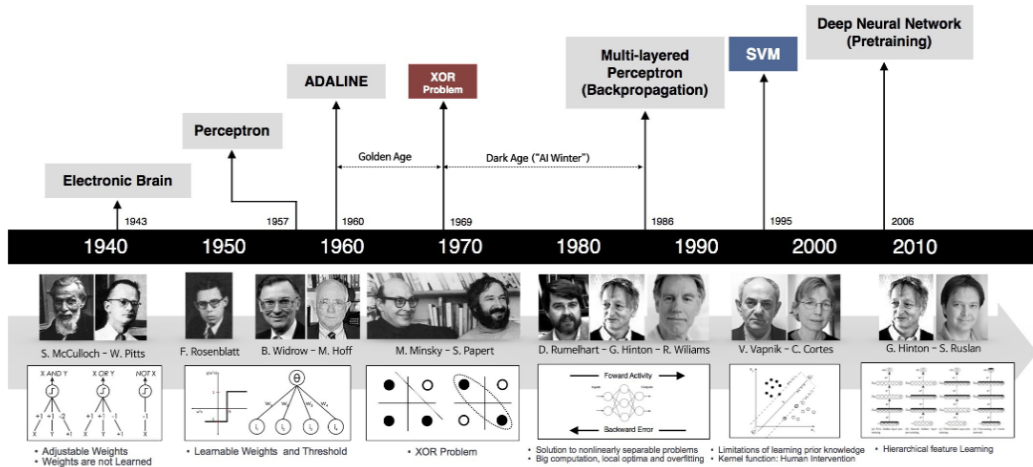
Original black-and-white frame



Automatic colorization using deep learning

Deep neural networks learn to predict plausible colors from grayscale input, trained on millions of color images.

# A Timeline of Deep Learning



# Deep Learning in Economics and Finance

## Forecasting & Nowcasting

- ▶ GDP nowcasting from high-frequency data
- ▶ Inflation forecasting with nonlinear models
- ▶ Yield curve modeling and term structure

## Text & Communication

- ▶ NLP on central bank minutes and speeches
- ▶ Sentiment analysis of financial news
- ▶ Measuring policy uncertainty

## Financial Stability & Supervision

- ▶ Fraud and anomaly detection
- ▶ Credit risk scoring
- ▶ Stress testing and scenario generation

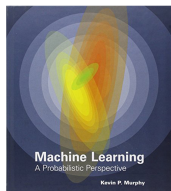
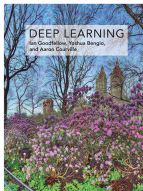
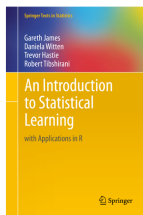
## Structural Models

- ▶ Solving and estimating structural models (DSGE, OLG, heterogeneous agents)
- ▶ Overcoming the curse of dimensionality
- ▶ NN-based surrogates for equilibrium computation

**This course** focuses on the last category, using deep learning as a computational tool for economic modeling.



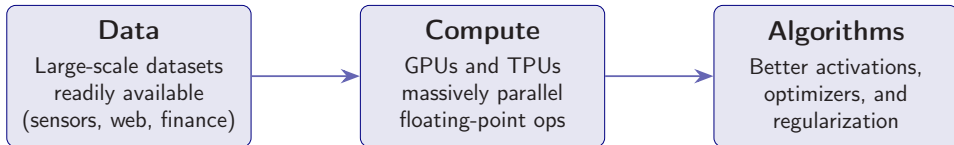
# Useful Materials



1. **I. Goodfellow, Y. Bengio, A. Courville** (2016). *Deep Learning*. MIT Press.  
<https://www.deeplearningbook.org>
2. **K. P. Murphy** (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
3. **G. James et al.** (2021). *An Introduction to Statistical Learning* (ISLR). Springer.  
<https://www.statlearning.com>

# Why Deep Learning Now?

Neural networks date back to the 1940s. Three developments made them practical:

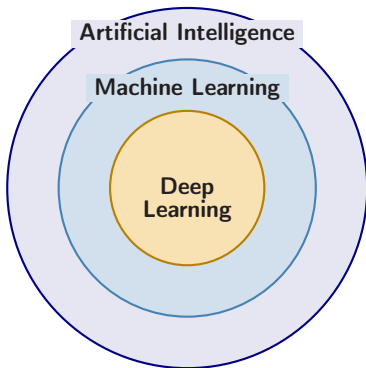


Key milestones: Perceptron (1958), Backpropagation (1986), Deep CNN (1995), AlexNet (2012), Transformers (2017), ChatGPT (2022).

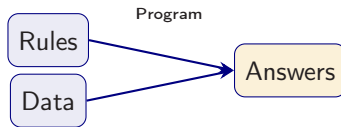
# Part I

## Machine Learning Foundations

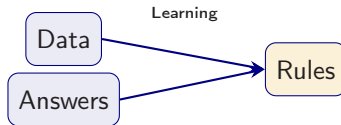
# What is Machine Learning?



## Classical programming:



## Machine learning:



# Types of Machine Learning

## Supervised

Given labeled pairs  $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ , learn a mapping  $h: \mathcal{X} \rightarrow \mathcal{Y}$ .

**Regression:**  $y \in \mathbb{R}$

**Classification:**

$y \in \{0, 1, \dots, K\}$

## Unsupervised

Given only  $\{\mathbf{x}^{(i)}\}_{i=1}^m$ , discover hidden structure.

**Clustering:** group similar points

**Dim. reduction:** compress features

## Reinforcement

An agent acts in an environment, receives rewards, and learns a policy  $\pi$ .

**Policy:**  $a_t = \pi(s_t)$

**Goal:** maximize  $\sum_t \gamma^t r_t$

This course covers **both** paradigms: we begin with the *supervised* pipeline (model, loss, optimizer)  
and then build on it for the *self-supervised residual* methods at the heart of the course (DEQNs, PINNs).

# Supervised Learning: Regression

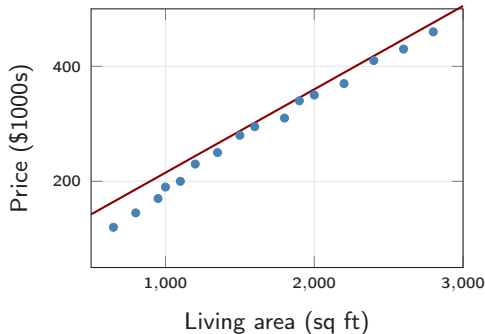
**Goal:** predict a continuous target  $y \in \mathbb{R}$  from features  $\mathbf{x} \in \mathbb{R}^d$ .

**Setup:**

- ▶ Training data:  $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$
- ▶ Model (hypothesis):  $\hat{y} = h(\mathbf{x}; \theta)$
- ▶ Parameters  $\theta$  learned from data

**Linear example:**

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



# Supervised Learning: Classification

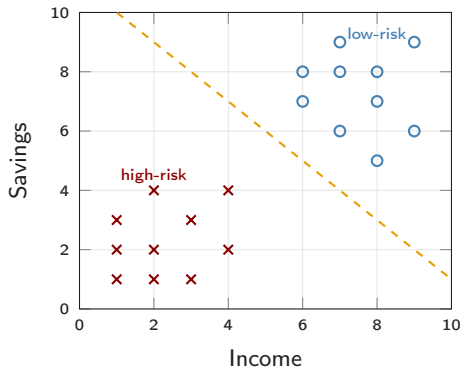
**Goal:** assign an input  $\mathbf{x}$  to one of  $K$  classes.

## Example – Credit scoring:

- Features: income, savings
- Labels: low-risk / high-risk
- Decision boundary separates classes

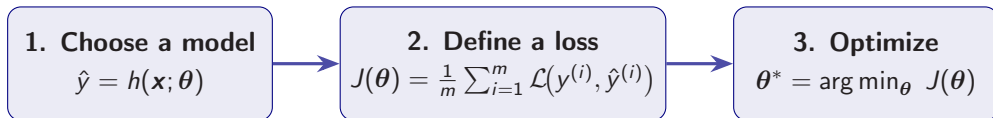
A simple linear rule:

$$\text{if } \mathbf{w}^T \mathbf{x} + b > 0 \Rightarrow \text{low-risk}$$



# The Machine Learning Recipe

Every machine learning algorithm follows the same three steps:



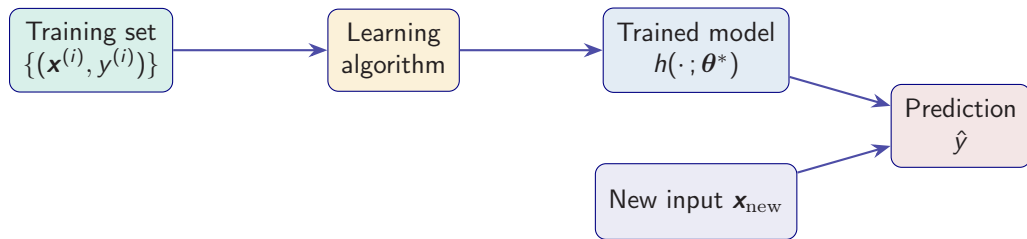
Cost function example (supervised regression: Mean Squared Error)

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

**Same recipe, different loss.** DEQNs and PINNs (later chapters) replace step 2 by an Euler-equation or PDE residual; steps 1 and 3 are unchanged.



# Supervised Learning Pipeline



Computationally, we need:

- ▶ Data (labeled examples)
- ▶ Linear algebra and automatic differentiation
- ▶ An optimization routine (gradient-based)

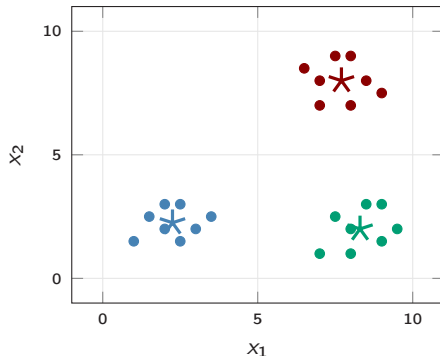
# Unsupervised Learning

No labels, only inputs  $\{\mathbf{x}^{(i)}\}_{i=1}^m$ . The goal is to discover hidden structure.

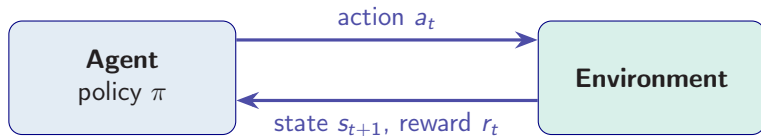
**Clustering:** partition data into groups of similar points (e.g., customer segmentation, image compression).

**Dimensionality reduction:** project high-dimensional data to a lower-dimensional space (e.g., PCA, autoencoders).

*k*-means clustering



# Reinforcement Learning (Brief)



- ▶ The agent learns a policy  $\pi(a | s)$  to maximize cumulative discounted reward  $\sum_{t=0}^{\infty} \gamma^t r_t$ .
- ▶ No explicit supervision, only delayed reward signals.
- ▶ Applications: game playing (AlphaGo), robotics, portfolio optimization.

We will not cover RL in this course, but the neural network tools you learn here are the building blocks for deep RL. Deep RL for sequential decision-making shares the Bellman structure developed in Chapters 2–5.

## Part II

### From Perceptrons to Deep Networks



# Biological Neurons



The **brain** is a massively parallel computer made of  $\sim 86$  billion neurons.

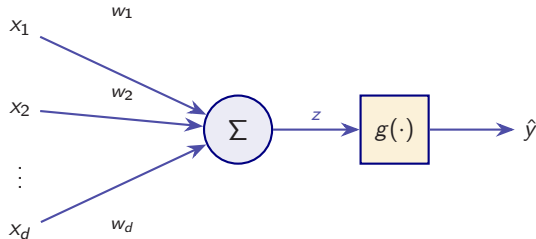
Each neuron:

- ▶ Receives signals via **dendrites**
- ▶ Integrates signals in the **cell body**
- ▶ Transmits output along the **axon**
- ▶ Communicates at **synapses**

Artificial neural networks are a *very loose* abstraction of this biological architecture.

# The Artificial Neuron (McCulloch & Pitts, 1943)

**From biology to math:** turn the neuron on the previous slide into an equation, synapses become weights  $w_i$ , dendritic inputs become  $x_i$ , the soma's integration becomes a sum, and firing becomes a nonlinear activation.



Three components:

1. **Weighted inputs**  $w_i$  (synaptic weights)
2. **Summation:**  $z = \sum_{j=1}^d w_j x_j$
3. **Activation**  $g(z)$ : nonlinear function determining whether the neuron “fires”

Original model used a step function as activation. Modern networks use smooth alternatives.

# Hebb's Rule (1949)

The missing ingredient — learning: the McCulloch–Pitts neuron is static; its weights  $w_i$  are fixed by hand. Hebb asks *how should the weights change with experience?*

**“Neurons that fire together, wire together.”**

Donald Hebb proposed that if neuron  $A$  repeatedly causes neuron  $B$  to fire, the synapse from  $A$  to  $B$  is strengthened:

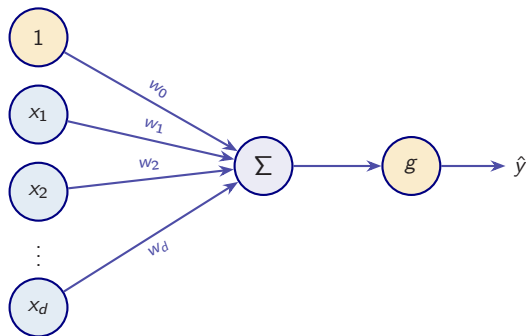
$$\Delta w_{ij} = \eta x_i y_j$$

This is the foundation of learning rules in artificial neural networks.

Analogy: every time you eat chocolate and feel happy, the association between “chocolate” and “happiness” neurons strengthens!



# The Perceptron



Forward pass:

$$\hat{y} = g \left( w_0 + \sum_{j=1}^d x_j w_j \right)$$

In vector notation, with  $\mathbf{w} = (w_1, \dots, w_d)^\top$ :

$$\hat{y} = g(w_0 + \mathbf{x}^\top \mathbf{w})$$

- ▶  $w_0$  is the **bias** (shifts activation)
- ▶  $g(\cdot)$  is a nonlinear **activation function**



# Forward Propagation – Vector Form

Absorbing the bias into the weight vector, define  $\tilde{\mathbf{x}} = (1, x_1, \dots, x_d)^\top \in \mathbb{R}^{d+1}$  and  $\tilde{\mathbf{w}} = (w_0, w_1, \dots, w_d)^\top \in \mathbb{R}^{d+1}$ . Then:

$$\hat{y} = g(\tilde{\mathbf{x}}^\top \tilde{\mathbf{w}})$$

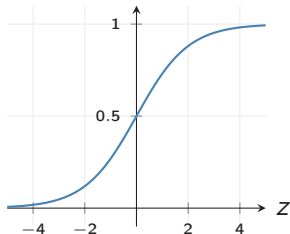
## Key insight

The perceptron computes a **weighted linear combination** of its inputs, applies a **nonlinearity**, and produces a scalar output. Without  $g$ , this is just linear regression.

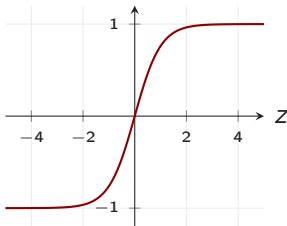
We will use the notation  $z = \mathbf{w}^\top \mathbf{x} + b$  (separating bias  $b$  from weights  $\mathbf{w}$ ) for the remainder of this lecture.

# Activation Functions

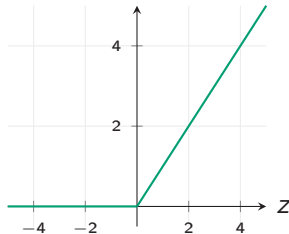
Sigmoid:  $\sigma(z) = \frac{1}{1+e^{-z}}$



Tanh:  $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$



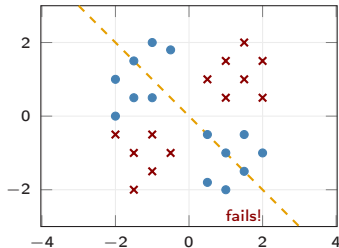
ReLU:  $\max(0, z)$



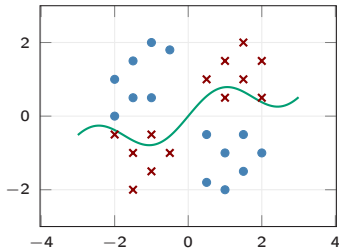
|         | Sigmoid                    | Tanh             | ReLU                                               |
|---------|----------------------------|------------------|----------------------------------------------------|
| Range   | $(0, 1)$                   | $(-1, 1)$        | $[0, \infty)$                                      |
| $g'(z)$ | $\sigma(z)(1 - \sigma(z))$ | $1 - \tanh^2(z)$ | $\begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$ |

# Why Nonlinearity Matters

**Linear activation**  $\Rightarrow$  linear boundary



**Nonlinear activation**  $\Rightarrow$  curved boundary



## Mathematical reason

A composition of linear maps is still linear:

$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{W}_2\mathbf{W}_1\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)$ . Without nonlinearities, depth is useless.

# Perceptron – A Worked Example

Suppose we have trained weights  $w_0 = 1$ ,  $\mathbf{w} = (3, -2)^\top$ , and sigmoid activation  $\sigma$ .

**Output:**

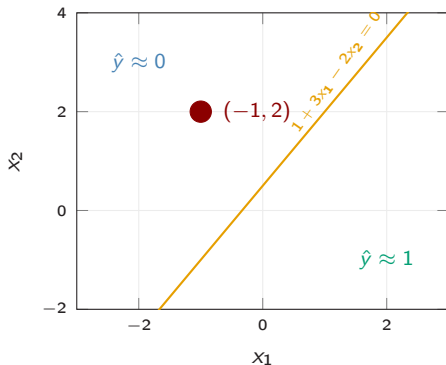
$$\hat{y} = \sigma(w_0 + \mathbf{x}^\top \mathbf{w}) = \sigma(1 + 3x_1 - 2x_2)$$

For a new point  $\mathbf{x} = (-1, 2)^\top$ :

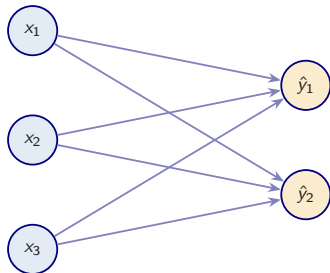
$$z = 1 + 3 \cdot (-1) - 2 \cdot 2 = -6$$

$$\hat{y} = \sigma(-6) \approx 0.0025$$

The discriminant line  $1 + 3x_1 - 2x_2 = 0$  separates the two classes.



# Multi-Output Perceptron



Each output unit computes its own weighted sum:

$$z_k = b_k + \sum_{j=1}^d w_{jk} x_j, \quad k = 1, \dots, K$$

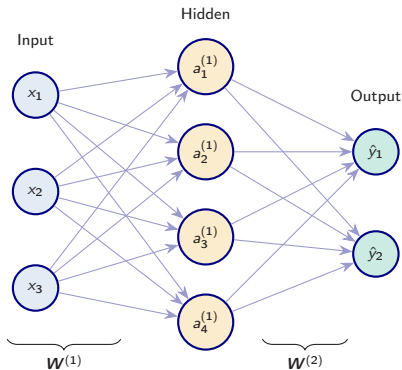
In matrix form:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad \hat{\mathbf{y}} = g(\mathbf{z})$$

where  $\mathbf{W} \in \mathbb{R}^{K \times d}$  (one row per output unit) and  $\mathbf{b} \in \mathbb{R}^K$ . The same row-per-output convention is used in every later layer of this deck, so the formula extends directly to deep networks (slides 23–24).

Each output has its own set of weights but shares the same inputs.

# Single Hidden Layer Network



Hidden layer:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{a}^{(1)} = g(\mathbf{z}^{(1)})$$

Output layer:

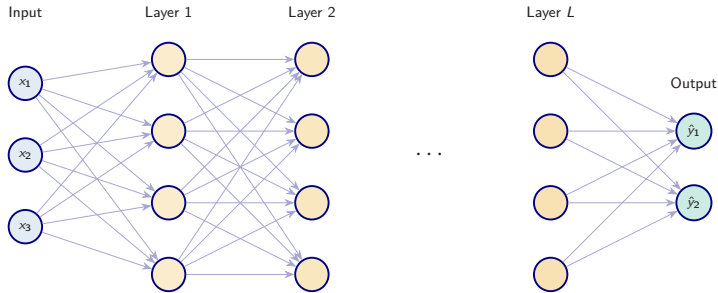
$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

$$\hat{\mathbf{y}} = g_{\text{out}}(\mathbf{z}^{(2)})$$

where  $\mathbf{W}^{(1)} \in \mathbb{R}^{n_1 \times d}$ ,  $\mathbf{W}^{(2)} \in \mathbb{R}^{K \times n_1}$ .

The hidden layer creates a **learned representation** of the input that the output layer then uses for prediction.

# Deep Feedforward Networks



An  $L$ -layer network computes a nested composition:

$$f(\mathbf{x}; \theta) = g_L \left( \mathbf{W}^{(L)} g_{L-1} \left( \cdots g_1 \left( \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)} \right) \cdots + \mathbf{b}^{(L-1)} \right) + \mathbf{b}^{(L)} \right)$$

Each layer transforms the representation. **Depth** provides exponential gains in representational power over width alone.

# Universal Approximation Theorem

## Theorem (Cybenko 1989; Hornik, Stinchcombe & White 1989)

Let  $g : \mathbb{R} \rightarrow \mathbb{R}$  be a bounded, non-constant, continuous activation function. Then for any continuous function  $f : \mathcal{K} \rightarrow \mathbb{R}$  on a compact set  $\mathcal{K} \subset \mathbb{R}^d$  and any  $\varepsilon > 0$ , there exists a single-hidden-layer network

$$F(\mathbf{x}) = \sum_{j=1}^{n_1} v_j g(\mathbf{w}_j^\top \mathbf{x} + b_j)$$

such that  $\sup_{\mathbf{x} \in \mathcal{K}} |F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$ .

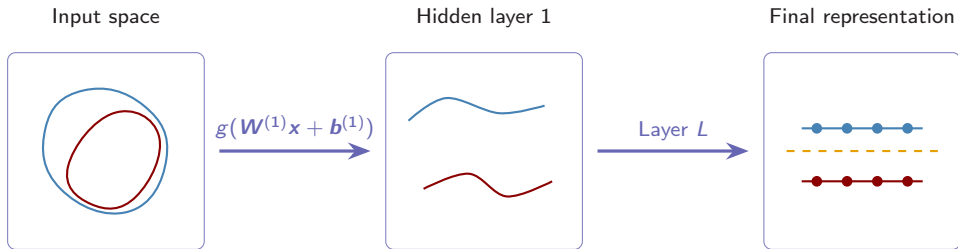
### Caveats:

- ▶ The required width  $n_1$  may be **exponentially large** in  $d$ .
- ▶ Existence  $\neq$  efficient learnability.
- ▶ Practical success of deep learning suggests that **depth** provides a much more efficient parameterization than width alone.



# A Geometric View: Learning to Untangle

Neural networks learn **successive coordinate transformations** that make the data linearly separable.



Analogy (Chollet, 2017): imagine crumpling two sheets of colored paper into a ball. A neural network learns how to smooth them out again, layer by layer, until a simple linear classifier can separate the colors.

# The Paper-Crumpling Analogy (Chollet, 2017)

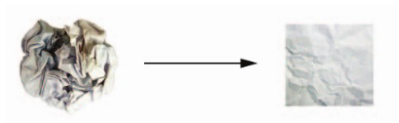


Figure 2.9 Uncrumpling a complicated manifold of data

Each layer of a deep network applies a simple geometric transformation:

- ▶ **Affine map**  $Wx + b$  (rotation, scaling, translation)
- ▶ **Activation**  $g(\cdot)$  (folding, warping)

Together, many layers “uncrumple” the data manifold until the classes become linearly separable.

# Part III

Training: Loss, Gradients, and Optimization

# Loss Functions

The loss function  $J(\boldsymbol{\theta})$  measures how far predictions  $\hat{\mathbf{y}}$  are from targets  $\mathbf{y}$ .

## Regression: Mean Squared Error

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta}))^2$$

Penalizes large deviations quadratically.

## Classification: Cross-Entropy

$$J = -\frac{1}{n} \sum_{i=1}^n \left[ y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$

For binary labels  $y \in \{0, 1\}$ .

**Multiclass** ( $K$  classes): use softmax output  $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$  and categorical cross-entropy:  $J = -\frac{1}{n} \sum_i \sum_k y_k^{(i)} \log \hat{y}_k^{(i)}$ .

# Beyond MSE: Robust and Asymmetric Losses

Let  $r = y - \hat{y}$  denote the residual.

**MAE (L1):** robust to outliers (constant gradient)

$$L_{\text{MAE}} = \frac{1}{n} \sum_{i=1}^n |r^{(i)}|$$

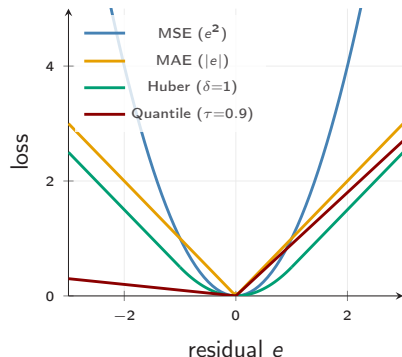
**Huber ( $\delta$ ):** smooth near zero, linear in the tails

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta(|r| - \frac{\delta}{2}) & |r| > \delta \end{cases}$$

**Quantile / Pinball ( $\tau \in (0, 1)$ ):**

$$L_{\tau}(r) = \begin{cases} \tau |r| & r \geq 0 \text{ (under-predict)} \\ (1-\tau) |r| & r < 0 \text{ (over-predict)} \end{cases}$$

$\tau > 0.5$ : penalises under-prediction more  $\Rightarrow$  predicts *high* quantile. Directly gives VaR at level  $\tau$ .



# Choosing a Loss Function

| Loss     | Outliers | Smoothness      | Symmetry          | Use case           |
|----------|----------|-----------------|-------------------|--------------------|
| MSE      | High     | Smooth          | Symmetric         | Default regression |
| MAE      | Low      | Non-smooth at 0 | Symmetric         | Median regression  |
| Huber    | Low      | Smooth          | Symmetric         | Robust regression  |
| Quantile | Low      | Non-smooth at 0 | <b>Asymmetric</b> | Risk, VaR, tails   |

## Why this matters in economics and finance

Tail risks are often more important than average performance. **Quantile loss** lets the model focus on getting worst-case predictions right, directly producing Value-at-Risk (VaR) or Expected Shortfall estimates. See **Notebook 07** for a hands-on comparison.

**References:** Huber (1964), Koenker & Bassett (1978).

# Gradient Descent – The Idea

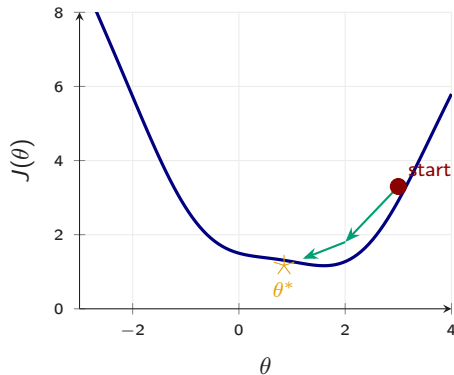
**Goal:** find weights that minimize the loss:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

**Strategy:** start from a random point, repeatedly step in the direction of steepest descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

where  $\eta > 0$  is the **learning rate**.



# Gradient Descent Algorithm

---

## Algorithm 1 \*

---

### Gradient Descent

- 1: Initialize  $\theta \sim \mathcal{N}(0, \sigma^2 I)$
  - 2: **repeat**
  - 3:   Compute gradient:  $\mathbf{g} \leftarrow \nabla_{\theta} J(\theta)$
  - 4:   Update weights:  $\theta \leftarrow \theta - \eta \mathbf{g}$
  - 5: **until** convergence
  - 6: **return**  $\theta$
- 

### Key considerations:

- ▶ Computing  $\nabla J$  over *all*  $n$  training points is  $O(n)$  per step, expensive for large datasets.
- ▶ The learning rate  $\eta$  controls step size.
- ▶ Convergence is to a **local** minimum (loss landscapes are non-convex for neural networks).



# Stochastic Gradient Descent (SGD)

**Idea:** approximate the full gradient with a cheap estimate from a small subset of data.

## Single-sample SGD

Pick one random example  $(\mathbf{x}^{(i)}, y^{(i)})$ :

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J_i(\boldsymbol{\theta})$$

Cheap but noisy.

## Mini-batch SGD

Sample a batch  $\mathcal{B}$  of  $B$  examples:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \frac{\eta}{B} \sum_{k \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} J_k(\boldsymbol{\theta})$$

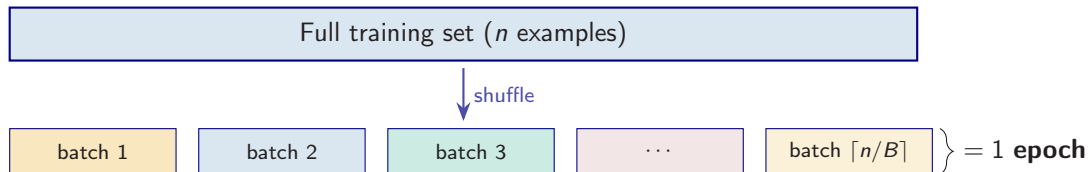
Reduces variance; enables GPU parallelism.

**Why does this work?** The mini-batch gradient is an **unbiased estimate** of the full gradient:

$$\mathbb{E} \left[ \frac{1}{B} \sum_{k \in \mathcal{B}} \nabla J_k \right] = \nabla J$$

Unbiased estimates suffice for convergence (Robbins & Monro, 1951).

# Mini-Batches and Epochs

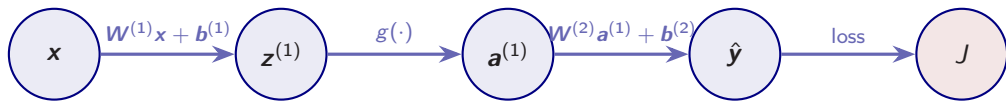


- ▶ One **epoch** = one complete pass through the training set.
- ▶ Typical batch sizes:  $B \in \{32, 64, 128, 256\}$ .
- ▶ Benefits of mini-batches:
  - Smoother convergence than single-sample SGD.
  - Efficient use of GPU parallelism.
  - Implicit regularization from gradient noise.

# Backpropagation – The Key Idea

**Problem:** how do we compute  $\frac{\partial J}{\partial \mathbf{W}^{(\ell)}}$  for each layer  $\ell$ ?

**Answer:** the **chain rule** of calculus, applied systematically.



$$\frac{\partial J}{\partial \mathbf{W}^{(1)}} = \underbrace{\frac{\partial J}{\partial \hat{y}}}_{\text{output error}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial a^{(1)}}}_{\mathbf{W}^{(2)}} \cdot \underbrace{\frac{\partial a^{(1)}}{\partial z^{(1)}}}_{g'(z^{(1)})} \cdot \underbrace{\frac{\partial z^{(1)}}{\partial \mathbf{W}^{(1)}}}_x$$

# Backpropagation – Forward Pass

Given input  $\mathbf{x}$ , compute layer by layer and store all intermediate quantities:

| Layer      | Pre-activation                                                             | Activation                                            |
|------------|----------------------------------------------------------------------------|-------------------------------------------------------|
| $\ell = 0$ |                                                                            | $\mathbf{a}^{(0)} = \mathbf{x}$                       |
| $\ell = 1$ | $\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{a}^{(0)} + \mathbf{b}^{(1)}$   | $\mathbf{a}^{(1)} = g(\mathbf{z}^{(1)})$              |
| $\ell = 2$ | $\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$   | $\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)})$              |
| $\vdots$   | $\vdots$                                                                   | $\vdots$                                              |
| $\ell = L$ | $\mathbf{z}^{(L)} = \mathbf{W}^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)}$ | $\hat{\mathbf{y}} = g_{\text{out}}(\mathbf{z}^{(L)})$ |

Finally compute the loss:  $J = \ell(\mathbf{y}, \hat{\mathbf{y}})$ . We need all the  $\mathbf{z}^{(\ell)}$  and  $\mathbf{a}^{(\ell)}$  values for the backward pass.

# Backpropagation – Backward Pass

Define the **error signal** at layer  $\ell$ :

$$\boldsymbol{\delta}^{(\ell)} \equiv \frac{\partial J}{\partial \mathbf{z}^{(\ell)}} \in \mathbb{R}^{n_\ell}$$

**Output layer** ( $\ell = L$ ):

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{z}^{(L)}} J = \frac{\partial J}{\partial \hat{\mathbf{y}}} \odot \mathbf{g}'_{\text{out}}(\mathbf{z}^{(L)})$$

**Hidden layers** ( $\ell = L-1, \dots, 1$ ) — propagate error *backward*:

$$\boxed{\boldsymbol{\delta}^{(\ell)} = (\mathbf{W}^{(\ell+1)})^\top \boldsymbol{\delta}^{(\ell+1)} \odot \mathbf{g}'(\mathbf{z}^{(\ell)})}$$

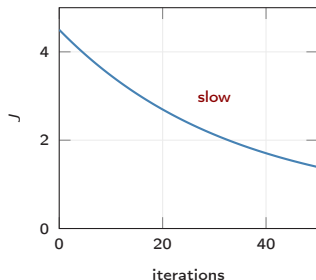
**Parameter gradients:**

$$\frac{\partial J}{\partial \mathbf{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} (\mathbf{a}^{(\ell-1)})^\top, \quad \frac{\partial J}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}$$

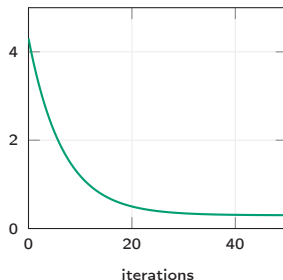
The symbol  $\odot$  denotes the element-wise (Hadamard) product.

# The Learning Rate

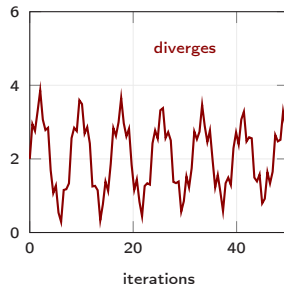
$\eta$  too small



$\eta$  just right



$\eta$  too large



- ▶ **Too small:** slow convergence, may get stuck in poor local minima.
- ▶ **Too large:** oscillation or divergence, overshooting the minimum.
- ▶ **Solution:** use **adaptive learning rates** that adjust automatically.

# Intermezzo – Hands-On

## Gradient Descent Notebook

Open and work through: `lectures/lecture_02_intro_deep_learning/code/lecture_02_02_GradientDescent_and`

**In this notebook, you will:**

1. Implement gradient descent from scratch for a simple function.
2. Compare full-batch GD with stochastic (per-sample) SGD.
3. Visualize how the learning rate affects convergence.

Log in and launch the “Deep Learning for Econ” application.

# Adaptive Optimizers

| Method          | Update rule                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Momentum</b> | $\mathbf{v}_t = \mu \mathbf{v}_{t-1} + \nabla J(\boldsymbol{\theta}_t); \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_t$                                                                                                                                                                                                                                                                                     |
| <b>RMSProp</b>  | $\mathbf{s}_t = \beta \mathbf{s}_{t-1} + (1 - \beta)(\nabla J)^2; \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \varepsilon}} \odot \nabla J$                                                                                                                                                                                                                                               |
| <b>Adam</b>     | $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla J$ (1st moment)<br>$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)(\nabla J)^2$ (2nd moment)<br>$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$ (bias correction)<br>$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon)$ |
| <b>AdamW</b>    | $\boldsymbol{\theta}_{t+1} = (1 - \eta\lambda)\boldsymbol{\theta}_t - \eta \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \varepsilon)$ (decoupled weight decay)                                                                                                                                                                                                                                                                |

**Adam** (Kingma & Ba, 2015) is the usual default ( $\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$ ). **AdamW** (Loshchilov & Hutter, 2019) decouples weight decay from the adaptive update, and is our default for PINN training in later chapters.



# Weight Initialization

**Why not initialize all weights to zero?** All neurons would compute the same output  
 $\Rightarrow$  same gradient  $\Rightarrow$  symmetry is never broken.

**Random initialization:** draw weights from a distribution scaled to the layer width  $n_\ell$   
(the number of neurons in layer  $\ell$ ;  $n_{\ell-1}$  is then the fan-in from the previous layer).

## Xavier / Glorot (2010)

For sigmoid or tanh activations:

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \frac{2}{n_{\ell-1} + n_\ell}\right)$$

Keeps activation variance roughly constant across layers.

## He (2015)

For ReLU activations:

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \frac{2}{n_{\ell-1}}\right)$$

Accounts for ReLU zeroing roughly half the variance.

Biases are typically initialized to zero. Proper initialization is critical for training deep networks.

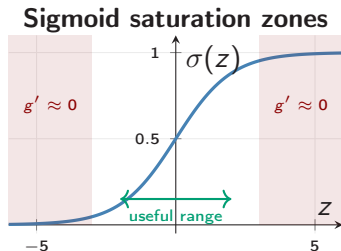
# Vanishing and Exploding Gradients

The backward pass multiplies through many layers:

$$\delta^{(\ell)} = (\mathbf{W}^{(\ell+1)})^\top \delta^{(\ell+1)} \odot g'(z^{(\ell)})$$

- ▶  $|g'(z)| < 1$  at every layer  $\Rightarrow$  product shrinks exponentially: **vanishing gradients**
- ▶  $|g'(z)| > 1$  or  $\|\mathbf{W}^{(\ell)}\|$  large  $\Rightarrow$  **exploding gradients**

**Remedies:** ReLU ( $g' \in \{0, 1\}$ ), proper initialization (He/Xavier), batch normalization, gradient clipping.



# Batch Normalization

**Idea** (Ioffe & Szegedy, 2015): normalize activations within each mini-batch to stabilize training.

For a mini-batch  $\mathcal{B} = \{z_1, \dots, z_B\}$  at some layer:

$$\mu_{\mathcal{B}} = \frac{1}{B} \sum_{i=1}^B z_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{B} \sum_{i=1}^B (z_i - \mu_{\mathcal{B}})^2, \quad \hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{z}_i + \beta$$

where  $\gamma$  and  $\beta$  are **learnable** scale and shift parameters.

## Benefits:

- ▶ Stabilizes training, allows higher learning rates.
- ▶ Mitigates vanishing/exploding activations across layers.
- ▶ Applied per-layer, typically before the activation function.

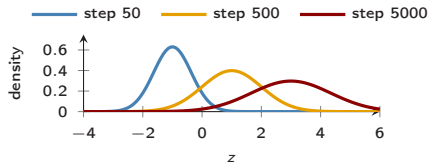
**Variants beyond batch.** LayerNorm, GroupNorm, and WeightNorm address the small-batch and recurrent settings; the script (§ Normalization Variants) treats them in detail.

# Batch Normalization: Intuition

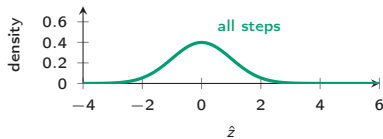
## Why does BN help?

- ▶ **The problem.** As earlier layers update, layer  $\ell$ 's input distribution *drifts* every step. Each layer chases a *moving target*  $\Rightarrow$  unstable gradients (*internal covariate shift*).
- ▶ **The fix.** BN = standardize pre-activations every mini-batch (just like standardizing regressors). Each layer always sees inputs at  $\mu=0, \sigma=1 \Rightarrow$  gradients flow predictably, larger learning rates become safe.
- ▶ **Why  $\gamma, \beta$ ?** They are learnable: a layer that prefers non-standard inputs can shift them back. BN never reduces capacity, it just gives optimization a better starting point.

## Pre-activations $z$ at one hidden layer

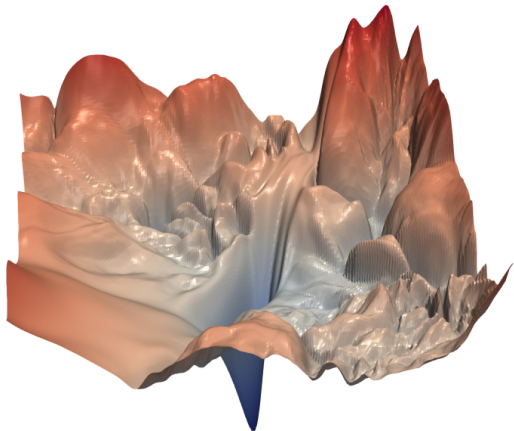


Without BN: distribution drifts  $\rightarrow$  moving target.



With BN: pinned to  $\mathcal{N}(0, 1)$  at every step.

# Loss Landscapes of Neural Networks



The loss surface of a deep network is highly non-convex:

- ▶ Many **local minima** and **saddle points**
- ▶ Sharp vs. flat minima affect generalization
- ▶ SGD noise helps escape sharp minima

Li et al. (2018): “Visualizing the Loss Landscape of Neural Nets.” Flat minima tend to generalize better than sharp ones.

# Modern Activation Functions: When to Use What

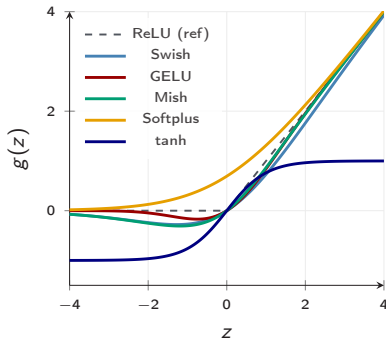
The common thread is **smoothness**. ReLU's kink at  $z=0$  causes *dead neurons* and an *undefined second derivative*. Two families fix this; they differ only in whether they keep ReLU's asymptotic shape.

**Smooth & ReLU-shaped** — default for feedforward / CNN / attention:

- ▶ **Swish/SiLU**  $z \sigma(z)$ : drop-in ReLU replacement (Ramachandran et al., 2017).
- ▶ **GELU**  $z \Phi(z)$ : transformer default (BERT, GPT, ViT).
- ▶ **Mish**  $z \tanh(\ln(1+e^z))$ : modern vision (YOLOv4).

**Smooth, different shape** — specialized outputs or differentiable losses:

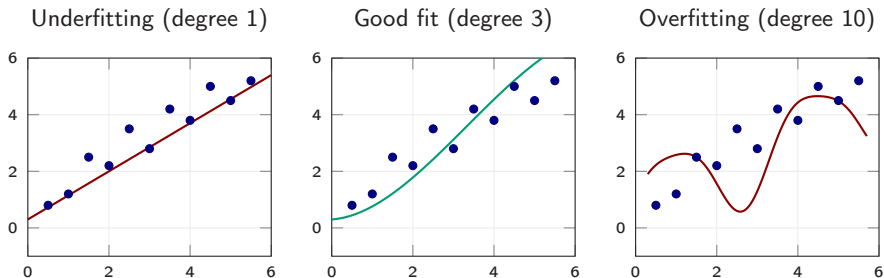
- ▶ **Softplus**  $\ln(1+e^z)$ : strictly positive  $\Rightarrow$  variance heads ( $\sigma^2$ ), Poisson rates.
- ▶ **tanh**: bounded  $C^\infty \Rightarrow$  PINNs (Day 6) where 2nd derivatives of the network enter the loss.



# Part IV

## Generalization and Regularization

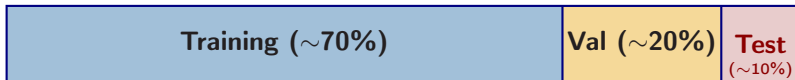
# Overfitting and Underfitting



- ▶ **Underfitting:** model too simple, high bias, cannot capture the pattern.
- ▶ **Overfitting:** model too complex, fits noise, poor generalization to new data.
- ▶ **Goal:** find the right complexity that generalizes well.



# Train / Validation / Test Split



## Training set

Used to fit model parameters  $\theta$  via optimization.

## Validation set

Used for model selection: hyperparameters, architecture, early stopping.

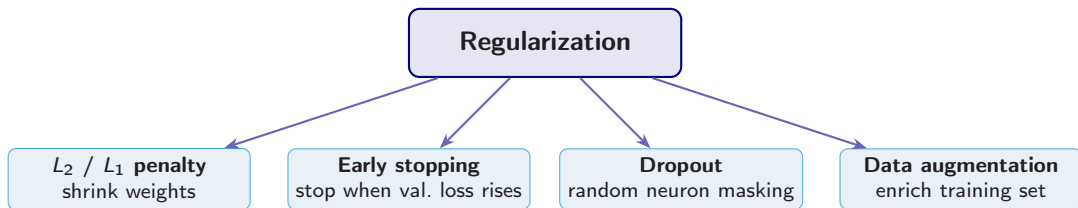
## Test set

Touched **once** at the very end to report final performance.

Never use the test set for any decision during model development!

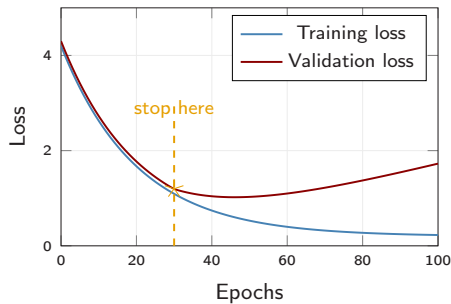
# Regularization Techniques — Overview

Goal: bias the learner toward **simpler** hypotheses so it generalizes beyond the training sample.



The next slides cover **early stopping**,  $L_2$ , and **dropout** in detail; data augmentation is task-specific (images, time series) and is left as a reading pointer.

# Early Stopping



**Idea:** monitor the validation loss during training. Stop when it starts increasing.

**In practice:**

1. After each epoch, evaluate loss on validation set.
2. Save model weights whenever validation loss improves.
3. If no improvement for  $p$  consecutive epochs (“patience”), stop training.
4. Return the saved best weights.

Early stopping is a simple and effective form of regularization.

# Regularization: $L_2$ Penalty and Dropout

## $L_2$ Regularization (Weight Decay):

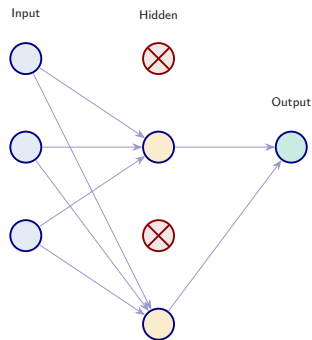
Add a penalty to the loss:

$$\tilde{J}(\theta) = J(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

Encourages small weights  $\Rightarrow$  simpler model.

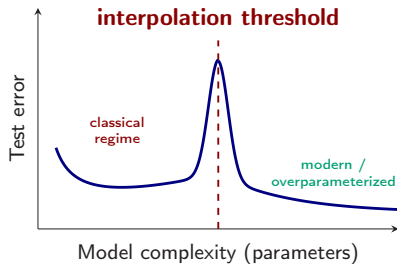
## Dropout (Srivastava et al., 2014):

During training, randomly zero out each neuron with probability  $p$  (typically 0.5). Forces the network not to rely on any single neuron. The standard *inverted dropout* variant divides activations by  $(1-p)$  during training, so no test-time rescaling is needed.



Dropout can be interpreted as training an **ensemble** of sub-networks that share weights.

# Deep Double Descent



The classical U turns into a **second descent** once the model is **overparameterized**. Three regimes along the curve:

1. **Classical** ( $n_{\text{par}} < n_{\text{data}}$ ): the familiar bias–variance U — small models underfit, larger ones chase noise.
2. **Interpolation peak** ( $n_{\text{par}} \approx n_{\text{data}}$ ): model *just* fits the training set; among all zero-error solutions the optimizer's pick is unstable  $\Rightarrow$  variance explodes.
3. **Modern** ( $n_{\text{par}} \gg n_{\text{data}}$ ): gradient descent implicitly picks the **minimum-norm / smoothest** interpolator  $\Rightarrow$  it generalizes *better*. More parameters help.

Belkin et al. (2019); Nakkiran et al. (2020).

**Hands-on:** `lectures/lecture_02_intro_deep_learning/code/lecture_02_03_Double_Descent.ipynb` — reproduce the curve with random Fourier features.

# Further Reading: NTK and Benign Overfitting

## Why does interpolation generalize? Two perspectives

- ▶ **Neural Tangent Kernel (NTK):** in the infinite-width limit, gradient descent on a neural network is equivalent to kernel regression with a fixed kernel; the kernel encodes the architecture.
  - ▶ Jacot, Gabriel & Hongler (2018, NeurIPS).
- ▶ **Benign overfitting:** interpolators generalize when signal aligns with the top eigendirections and noise spreads across many small ones.
  - ▶ Bartlett, Montanari & Rakhlin (2021, *Proc. Natl. Acad. Sci.*).

These theories provide a theoretical foundation for the double-descent phenomenon. Both are active research areas; consult the course reading list for entry points.

# Part V

## Advanced Architectures & Frontiers

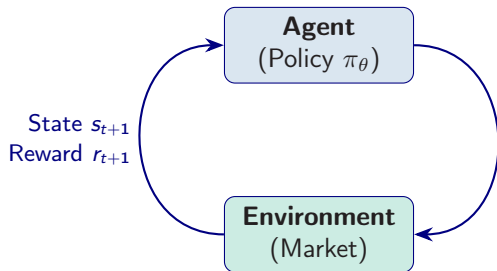
# Outlook (not covered): RL in Finance

## The RL Loop:

- ▶ **Agent:** Portfolio manager / Algo trader
- ▶ **Environment:** The market (prices, order book)
- ▶ **State  $s_t$ :** Current prices, portfolio weights
- ▶ **Action  $a_t$ :** Buy/sell, rebalance
- ▶ **Reward  $r_t$ :** P&L, Sharpe ratio, utility

## Why not in this course?

- ▶ RL finds *strategies* in a *fixed* environment.
- ▶ We focus on *finding the equilibrium* (the environment itself).
- ▶ BUT: critical for hedging & execution.





# Why Sequences? Memory Matters

Many real problems are intrinsically sequential:

- ▶ **Next-word prediction:** “Today we are having a class on deep \_\_\_\_\_”
- ▶ **Time-series forecasting:** inflation, volatility, returns conditional on history
- ▶ **Audio / music:** each sample predicted from thousands before (MuseNet, Wav2Vec)
- ▶ **Econ application:** Edgeworth cycles in gasoline prices (asymmetric sawtooth)

Why an MLP is not enough.

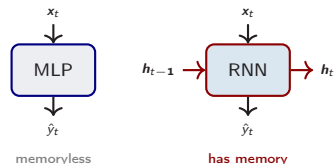
- ▶ MLP output is *memoryless*:  $\hat{y}_t = f(x_t)$ .
- ▶ Same input  $\Rightarrow$  same output, regardless of history.
- ▶ Sequences have *variable length*; MLPs have a fixed input size.
- ▶ We need a network that **carries state across time**.

**Design goals** (Elman 1990; Rumelhart, Hinton & Williams 1986): **variable length** · **order** · **long-range dependence** · **parameter sharing across time**.

# From MLP to Recurrence

**Idea.** Give the cell a persistent **hidden state**  $\mathbf{h}_t \in \mathbb{R}^d$ , fed back at every step.

- ▶ At step  $t$ , the cell sees *both* the new input  $\mathbf{x}_t$  and yesterday's state  $\mathbf{h}_{t-1}$ .
- ▶ Same parameters  $\{\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}\}$  at every step:  
**parameter sharing across time.**
- ▶ The hidden state  $\mathbf{h}_t$  is a learned summary of the past, the network's "memory".
- ▶ *One* cell processes *any* sequence length.



**RNN is a dynamical system.** Not a static function.

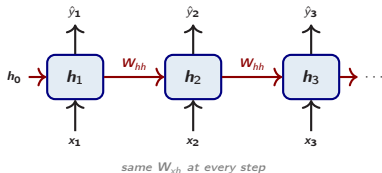
# The RNN Cell, Unrolled Across Time

Two equations define the cell:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$

$$\hat{y}_t = W_{hy} h_t + b_y$$

- ▶  $h_0$  initialized to 0 (or learned).
- ▶ **Weight sharing:**  $W_{hh}$ ,  $W_{xh}$ ,  $W_{hy}$  do not depend on  $t$ .
- ▶ A *folded* loop with  $T$  steps  $\equiv$  an *unfolded* feedforward graph of depth  $T$  with *shared* parameters.



"Same function, many steps." Elman (1990); Rumelhart, Hinton & Williams (1986).

# Training: Backpropagation Through Time (BPTT)

**Idea.** The unrolled RNN is a feedforward net of depth  $T$  with *shared* weights. Run backprop, *sum* weight-gradients across time.

**Gradient through time.** For the gradient of  $\mathcal{L}_T$  w.r.t. an early state  $\mathbf{h}_k$ ,

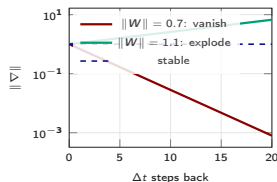
$$\nabla_{\mathbf{h}_k} \mathcal{L}_T = \left( \prod_{t=k+1}^T \underbrace{\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}}_{= \text{diag}(\tanh') \mathbf{W}_{hh}} \right)^\top \nabla_{\mathbf{h}_T} \mathcal{L}_T.$$

Standard Jacobian convention; the transpose rides on the chain-rule product. Details in the script's §1.10 “Vanishing and Exploding Gradients”.

A **long product of the same Jacobian**:

- ▶ If  $\|\mathbf{W}_{hh}\| > 1$ : gradient **explodes**  $\rightarrow$  NaN.
- ▶ If  $\|\mathbf{W}_{hh}\| < 1$ : gradient **vanishes**  $\rightarrow$  no learning of long-range structure.

**Remedies:** gradient clipping, orthogonal init, truncated BPTT, and above all **gated cells** (LSTM, GRU).



# LSTM: A Memory Conveyor Belt

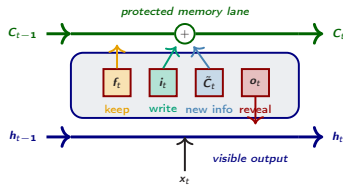
**Plain-language picture.** A vanilla RNN rewrites its memory at every step. An LSTM keeps a dedicated **cell state**  $C_t$  that mostly flows forward unchanged.

At each date, the cell makes three soft decisions:

- **Forget gate**  $f_t$ : how much of the old memory survives?
- **Input gate**  $i_t$ : how much new information should be written?
- **Output gate**  $o_t$ : how much of memory should be exposed as  $h_t$ ?

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh(C_t)$$

- **Key win:** memory changes *additively*, so useful information can survive many steps.
- Useful for persistent asymmetric patterns, e.g. Edgeworth cycles.



# Limits of Recurrence $\Rightarrow$ Attention

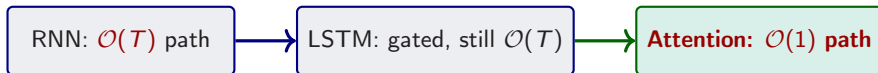
Even with LSTMs, two hard limits remain.

## 1. Sequential $\Rightarrow$ no parallelism.

- ▶ Step  $t$  must wait for step  $t-1$ .
- ▶ GPUs love parallelism  $\Rightarrow$  RNN training is memory-bound and slow.
- ▶ Training modern LLMs on  $\sim 1T$  tokens with RNNs would take *years*.

## 2. Path length blurs long-range signals.

- ▶ Information from position 1 reaches position  $T$  through  $T-1$  hops.
- ▶ Each hop applies a matrix + nonlinearity  $\Rightarrow$  fading memory.
- ▶ Gating helps but does not eliminate path-length decay.



**Key move.** Drop recurrence entirely: let every position read every other *directly*, in *parallel*. This is the **Transformer** (Vaswani et al. 2017).

# Self-Attention: Search Inside the Sequence

**Mental model.** Updating a token is a search over the whole sequence.

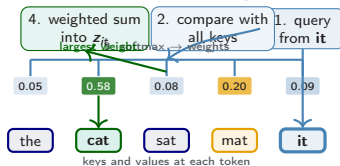
**Each token creates three vectors:**

- **Query**  $q_i$ : what am I looking for?
- **Key**  $k_j$ : what does token  $j$  advertise?
- **Value**  $v_j$ : what can token  $j$  contribute?

**Four-step recipe:**

1. Add position information.
2. Compare  $q_i$  with every key  $k_j$ .
3. Softmax the scores into weights  $\alpha_{ij}$ .
4. Return the weighted average of the values.

## Worked attention pattern



Blue arrows: build the query and compare it to all keys.  
Green arrows: use the weights to pull information, mainly from “cat”, into  $z_{it}$ .

Displayed weights:

$$0.05 + 0.58 + 0.08 + 0.20 + 0.09 = 1.00.$$

# From Self-Attention to a Transformer Block

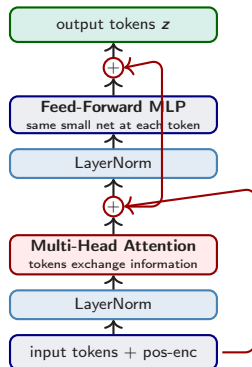
**Matrix form:**

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

**Why the extra ingredients?**

- **Multi-head attention:** different heads can focus on different relations in parallel.
- **Positional encoding:** attention alone cannot see order, so we add a ruler.
- **Residuals + LayerNorm + MLP:** make deep stacks trainable and expressive.

**Econ gloss:** learnable kernel smoothing with learned features and learned similarity.



**One block** first mixes information *across tokens*, then transforms each token with an MLP, while skip connections keep deep stacks trainable.



# RNN, LSTM, Transformer: What to Remember

## RNN

- ▶ Hidden state = running summary of the past
- ▶ Best first mental model for sequence learning
- ▶ Hard to train on long histories

## LSTM / GRU

- ▶ Add gates and a protected memory lane
- ▶ Strong baseline for specialized time-series tasks
- ▶ Still sequential, so no large-scale parallelism

## Transformer

- ▶ Every token can read every other token directly
- ▶ Full parallelism across positions
- ▶ Default choice for long-context sequence modeling

## Rule of Thumb

If the pattern is specialized and the history is moderate, an LSTM is still a strong baseline. If context is long and parallelism matters, start with a Transformer.

Earlier-lecture notebooks: 08\_MLP\_LSTM\_Transformer\_Edgeworth\_Cycles (the three-way comparison on the same task) and 09\_Transformer\_InContext\_AR1 (advanced / optional).

# Part VI

Software, Exercises, and Next Steps

# Software Ecosystem

## TensorFlow

+ Keras

Google, 2015

## PyTorch

Meta AI, 2017

## JAX

Google, 2018

- ▶ Keras (<https://keras.io>) provides a high-level API that runs on top of TensorFlow, PyTorch, or JAX—write once, run anywhere.
- ▶ Key resources:
  - ▶ Keras sequential model: [https://www.tensorflow.org/guide/keras/sequential\\_model](https://www.tensorflow.org/guide/keras/sequential_model)
  - ▶ TensorBoard: <https://www.tensorflow.org/tensorboard>
  - ▶ PyTorch tutorials: <https://pytorch.org/tutorials>

This chapter exercises both stacks: notebooks 04, 05, 07 use **TensorFlow/Keras**; notebooks 06, 08, 09 use **PyTorch**. Later DEQN/PINN material is TensorFlow-based; the heterogeneous-agent and sequence-space chapters add JAX.

# Notation Pact for the Rest of the Course

This chapter uses  $\theta$  for generic trainable model parameters, following the textbook convention. From Chapter 2 onward the script switches:

- ▶  $\rho$  = neural-network weights (e.g.  $\mathcal{N}_\rho(\mathbf{x})$ ), matching Azinovic, Gaegauf & Scheidegger (2022) and the rest of the DEQN literature.
- ▶  $\theta$  = *structural* economic parameters that the network does not see (e.g. Chs. 10 & 11 on structural estimation and climate).

Both symbols denote the same object *abstractly* (free parameters of the model); the rename foregrounds which kind of parameter is being trained. Whenever a network appears in subsequent chapters, expect  $\rho$ .

# Hands-On Notebooks

The companion Jupyter notebooks are available in the course repository:

| # | Notebook                                 | Content                                                       |
|---|------------------------------------------|---------------------------------------------------------------|
| 1 | 01_BasicML_intro                         | Linear regression, classification, clustering, loss functions |
| 2 | 02_GradientDescent_and_SGD               | Gradient descent, line search, SGD                            |
| 3 | 03_Double_Descent                        | Double-descent with random Fourier features                   |
| 4 | 04_Gentle_DNN                            | First neural network with Keras                               |
| 5 | 05_Tensorboard                           | Visualizing training with TensorBoard                         |
| 6 | 06_PyTorch_intro                         | Introduction to PyTorch                                       |
| 7 | 07_Genz_Approximation_and_Loss           | Genz functions, NN approximation, robust losses               |
| 8 | 08_MLP_LSTM_Transformer_Edgeworth_Cycles | MLP vs LSTM vs tiny Transformer on Edgeworth cycles           |
| 9 | 09_Transformer_InContext_AR1             | Tiny transformer, in-context learning of AR(1)                |

File #2: 02\_GradientDescent\_and\_StochasticGradientDescent.ipynb; #7: 07\_Genz\_Approximation\_and\_Loss\_Functions.ipynb.

# Exercise: Function Approximation with Neural Networks

**Task:** approximate the Genz (1987) test functions with neural networks.

|                                                                                                                                                          |                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| $f_1(\mathbf{x}) = \cos\left(2\pi w_1 + \sum_{i=1}^d c_i x_i\right)$                                                                                     | (oscillatory)   |
| $f_2(\mathbf{x}) = \prod_{i=1}^d (c_i^{-2} + (x_i - w_i)^2)^{-1}$                                                                                        | (product peak)  |
| $f_3(\mathbf{x}) = \left(1 + \sum_{i=1}^d c_i x_i\right)^{-(d+1)}$                                                                                       | (corner peak)   |
| $f_4(\mathbf{x}) = \exp\left(-\sum_{i=1}^d c_i^2 (x_i - w_i)^2\right)$                                                                                   | (Gaussian)      |
| $f_5(\mathbf{x}) = \exp\left(-\sum_{i=1}^d c_i  x_i - w_i \right)$                                                                                       | (continuous)    |
| $f_6(\mathbf{x}) = \begin{cases} 0 & \text{if } x_1 > w_1 \text{ or } x_2 > w_2 \\ \exp\left(\sum_{i=1}^d c_i x_i\right) & \text{otherwise} \end{cases}$ | (discontinuous) |

Pick 3 functions. Train NNs on 10, 50, 100, 500 random points from  $[0, 1]^d$ . Compute average and max error on 1000 test points. Repeat for  $d \in \{2, 5, 10\}$ .

# Exercise: Architecture Exploration

Using the Genz functions from the previous exercise:

1. **Vary the depth:** try 1, 2, 3, and 5 hidden layers. How does depth affect approximation quality?
2. **Vary the activation:** compare sigmoid, tanh, ReLU, and Swish. Which works best for smooth vs. discontinuous functions?
3. **Vary the optimizer:** compare plain SGD, SGD with momentum, and Adam. Monitor convergence speed and final accuracy.
4. **Plot** the maximum and average approximation error as a function of the number of training points. What happens as  $d$  grows?

**Hint:** you should observe the “curse of dimensionality”, the error grows significantly with  $d$  for a fixed number of training points.

# Questions?



Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: [https://github.com/sischei/Deep\\_Learning\\_for\\_Solving\\_And\\_Estimating\\_Dynamic\\_Economic\\_Models](https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models)